

Arsitektur Pengembangan Perangkat Lunak Berbasis Kecerdasan Buatan: Formulasi Spesifikasi Dokumen (PRD, FSD, TSD) dan Analisis Alur Kerja Komprehensif

Pengantar Paradigma Pengembangan Berbasis Spesifikasi

Siklus hidup pengembangan perangkat lunak (Software Development Life Cycle - SDLC) saat ini sedang mengalami pergeseran paradigma yang sangat fundamental. Industri bergerak menjauh dari model pembuatan sintaksis kode secara manual menuju model pengawasan arsitektural yang digerakkan secara penuh oleh kecerdasan buatan (Artificial Intelligence - AI). Dalam ekosistem yang berkembang pesat ini, peran operator manusia berevolusi dari seorang pemrogram tradisional (programmer) menjadi manajer produk, perencana sistem, atau arsitek perangkat lunak tingkat tinggi. Fungsi utama dari pengembang tidak lagi menulis baris demi baris kode logika, melainkan melakukan perencanaan konseptual tingkat tinggi, mengorkestrasi berbagai model bahasa besar (Large Language Models - LLM), serta melakukan tinjauan teknis dan tinjauan kualitas akhir.

Evolusi metodologi ini sangat bergantung pada konsep "Rekayasa Konteks" (Context Engineering). Rekayasa konteks adalah sebuah pendekatan di mana dokumentasi yang sangat terstruktur, kaku, dan deterministik digunakan untuk mendikte keluaran dari agen kecerdasan buatan. Tujuan utamanya adalah meminimalkan penyimpangan logika dan secara proaktif mencegah fenomena yang umumnya dikenal sebagai "halusinasi AI". Agar agen AI dapat secara otonom membangun aplikasi kompleks skala penuh (full-stack application), agen tersebut memerlukan jangkar konseptual yang tidak dapat diubah.

Jangkar konseptual ini diwujudkan dalam bentuk tiga dokumen spesifikasi utama: *Project Requirements Document* (PRD), *Functional Specification Document* (FSD), dan *Technical Specification Document* (TSD). Alih-alih memanfaatkan dokumen-dokumen ini sekadar sebagai alat komunikasi antarmanusia (seperti dalam metodologi Agile atau Waterfall tradisional), alur kerja AI modern memanfaatkan dokumen-dokumen ini sebagai "sumber kebenaran" (source of truth) yang bersifat programatik. Dokumen-dokumen tersebut ditanamkan langsung ke dalam ruang kerja (workspace) agen AI, biasanya direpresentasikan sebagai file Markdown fisik (misalnya, prd.md, summary.md, atau api-contract.md), yang memastikan bahwa model bahasa besar beroperasi dalam jendela kontekstual yang didefinisikan secara ketat dan

konsisten.

Laporan ini menyajikan analisis yang sangat mendalam, terperinci, dan bernuansa mengenai alur kerja pengkodean AI yang optimal. Analisis ini mencakup rantai alat (toolchains) yang spesifik, konvensi gaya pengkodean, dan formulasi dokumen perencanaan strategis yang diperlukan untuk mengorkestrasi pembangunan perangkat lunak yang kompleks. Selanjutnya, laporan ini menyaring metodologi operasional tersebut menjadi rancangan cetak biru (blueprint) yang kuat dan dapat ditindaklanjuti untuk pembuatan PRD, FSD, dan TSD. Rancangan ini secara eksplisit dirancang untuk digunakan sebagai instruksi (prompt) utama bagi berbagai agen AI, memungkinkan sinkronisasi konteks yang presisi di berbagai tugas rekayasa perangkat lunak.

Filosofi Rekayasa Konteks dalam Ekosistem AI

Mekanisme fundamental yang mendasari keberhasilan pengembangan perangkat lunak yang digerakkan oleh AI adalah rekayasa konteks yang disiplin. Mode kegagalan yang paling umum dalam pemrograman berbantuan AI tahap awal terjadi ketika pengembang mencoba menghasilkan seluruh aplikasi melalui instruksi tunggal yang monolitik (sering disebut sebagai "zero-shot prompting"). Pendekatan yang naif ini secara konsisten menghasilkan kehilangan konteks (context loss) yang sangat fatal. Ketika kapasitas token model bahasa tercapai atau terlampaui, model tersebut akan mulai melupakan persyaratan struktural awal, mencampurkan paradigma desain yang tidak kompatibel, menciptakan fungsi pustaka (library) yang tidak ada, atau kehilangan jejak status aplikasi secara keseluruhan.

Untuk menghindari kegagalan sistemik ini, metodologi yang berlaku saat ini memberlakukan pendekatan *Spec-Driven Development* (Pengembangan Berbasis Spesifikasi). Dalam model ini, pengembang manusia dan agen AI terlibat dalam fase perencanaan yang kolaboratif dan berulang untuk menyusun spesifikasi yang komprehensif sebelum satu baris kode yang dapat dieksekusi ditulis oleh mesin.

Doktrin "Sumber Kebenaran" (Source of Truth)

Alur kerja berbasis AI yang stabil sangat bergantung pada pembentukan "sumber kebenaran" yang permanen di dalam Lingkungan Pengembangan Terpadu (Integrated Development Environment - IDE). Proses ini dimulai dengan bertukar pikiran (brainstorming) menggunakan model AI dengan kemampuan penalaran tinggi. Ide-ide mentah tersebut kemudian dikompilasi menjadi dokumen berformat Markdown yang terstruktur (seperti `prd.md` atau `summary.md`). Dokumen ini menciptakan basis pengetahuan yang persisten di dalam sistem file lokal.

Keuntungan sekunder dan tersier dari doktrin sumber kebenaran ini sangat signifikan. Ketika pengembang perlu menginstruksikan agen AI yang sedang mengeksekusi kode, mereka tidak perlu lagi mengulangi parameter proyek secara lengkap. Sebaliknya, instruksi (prompt) menjadi sangat modular dan referensial. Sebagai contoh, pengembang cukup menginstruksikan: "Bangun logika backend dengan mematuhi secara ketat skema yang telah didefinisikan dalam file `prd.md`".

Metodologi ini secara drastis mengurangi biaya komputasi dan overhead token, sekaligus memaksakan konsistensi struktural. Ketika model AI memiliki akses konstan ke narasi proyek yang menyeluruh melalui file Markdown tersebut, probabilitas AI memperkenalkan ketidakkonsistenan logis di berbagai lapisan arsitektur (misalnya, ketidakcocokan antara antarmuka pengguna frontend dan titik akhir API backend) akan mendekati angka nol.

Mengatasi Batasan Pengetahuan (Knowledge Cutoff) dengan Model Context Protocol

Tantangan kedua yang paling kritis dalam rekayasa konteks adalah tanggal batas pengetahuan (knowledge cutoff date) intrinsik dari LLM yang telah dilatih sebelumnya (pre-trained). Ekosistem kerangka kerja (framework) perangkat lunak berevolusi dengan kecepatan yang sangat eksponensial. Sebuah model bahasa yang dilatih pada kumpulan data hingga akhir tahun 2023 akan dengan penuh percaya diri menghasilkan sintaksis yang sudah usang (deprecated) untuk kerangka kerja yang dirilis atau diperbarui secara masif pada tahun 2025. Keyakinan algoritmik yang salah ini menyebabkan waktu debugging yang berlarut-larut.

Untuk memecahkan kebuntuan arsitektural ini, alur kerja kecerdasan buatan tingkat lanjut memanfaatkan penerapan standar *Model Context Protocol* (MCP). MCP berfungsi sebagai jembatan dinamis dan terstandarisasi antara agen AI yang terisolasi dengan sumber data eksternal yang bersifat waktu nyata (real-time). Dalam ekosistem ini, alat bantu seperti Context7 bertindak sebagai server MCP, memanfaatkan teknik *Retrieval-Augmented Generation* (RAG) untuk mengambil dokumentasi yang paling mutakhir untuk pustaka atau kerangka kerja apa pun yang sedang digunakan.

Sebelum AI agen mulai menghasilkan rencana implementasi, model tersebut diinstruksikan secara eksplisit untuk menanyakan kepada server MCP, menelan dokumentasi terbaru (termasuk titik akhir API baru dan aturan sintaksis terbaru), dan memformat kodenya dengan tepat. Proses injeksi konteks ini mengubah agen AI dari repositori data historis yang statis menjadi konsumen aktif dokumentasi teknis terkini, yang secara efektif meniadakan halusinasi berbasis versi.

Ekosistem Konfigurasi dan Orkestrasi Alat (Toolchain)

Pelaksanaan alur kerja yang digerakkan oleh spesifikasi (spec-driven) tidak dapat mengandalkan antarmuka perpesanan sederhana seperti peramban obrolan web standar. Analisis menunjukkan bahwa ekosistem ini memerlukan konfigurasi berlapis yang sangat terspesialisasi. Pengembang bertindak sebagai konduktor, mengorkestrasi serangkaian alat, IDE yang terintegrasi, dan model AI yang ditargetkan untuk tugas spesifik.

Lingkungan Pengembangan Berbasis Agen (Agent-Based IDEs)

Arena utama untuk pengembangan berbasis AI telah beralih sepenuhnya dari editor teks tradisional menuju lingkungan yang dirancang secara native untuk agen kecerdasan buatan. Lingkungan ini harus memiliki kemampuan membaca dan menulis langsung ke sistem file lokal

pengguna. Salah satu alat yang paling menonjol dalam alur kerja ini adalah lingkungan kolaboratif bernama Anti Gravity—sebuah IDE atau pembungkus (wrapper) AI (sering kali merupakan turunan dari VS Code) yang memberikan otonomi eksekusi file kepada model bahasa.

Di dalam lingkungan agen seperti Anti Gravity, pengembang menerapkan beberapa mode eksekusi yang sangat terstruktur:

- 1. **Agent Manager / Mode Tampilan Agen (Agent View):** Ini adalah antarmuka interaksi utama dan terus-menerus tempat pengembang berkomunikasi dengan model. Antarmuka ini dirancang untuk mengelola sesi pengkodean yang berjalan lama tanpa kehilangan rangkaian konteks awal.
- 2. **Mode Perencanaan (Plan Mode):** Ini adalah keadaan paling krusial sebelum penulisan sintaksis. Dalam mode ini, AI menganalisis dokumentasi spesifikasi (FSD/PRD) yang disediakan dan mengeluarkan peta jalan implementasi yang diusulkan. Operator manusia meninjau, mengomentari, dan akhirnya menyetujui rencana ini secara sebaris (inline) sebelum mengeksekusinya.

Sebagai pelengkap, alat lain seperti Zed.dev sering dipilih karena kinerjanya yang jauh lebih ringan dibandingkan dengan IDE monolitik konvensional. Selain itu, Antarmuka Baris Perintah (Command Line Interfaces - CLI) berbasis AI seperti Droid CLI dan Codex CLI sangat esensial. CLI digunakan secara strategis untuk mengabaikan batasan laju (rate limits) dari IDE utama, atau untuk mendelegasikan tugas-tugas latar belakang yang terisolasi—seperti membuat konfigurasi containerisasi Docker atau menjalankan perpindahan basis data—kepada model AI sekunder sementara model primer berkonsentrasi pada rekayasa antarmuka pengguna.

Strategi Perutean Model (Model Routing Strategy)

Karakteristik yang paling mendefinisikan dari alur kerja AI tingkat ahli adalah penghindaran ketergantungan pada satu LLM tunggal. Sebagai gantinya, metodologi ini mengadopsi "strategi perutean model" (model routing), di mana tugas-tugas mikro didelegasikan kepada model kecerdasan buatan yang secara matematis dan arsitektural paling sesuai untuk tugas tersebut. Akses ke keragaman model ini sering kali difasilitasi melalui penyedia perutean API seperti OpenRouter, atau melalui kemampuan bawaan IDE agen.

Tugas Spesifik Ekosistem Perangkat Lunak	Model AI yang Direkomendasikan	Rasionalisasi & Karakteristik Komputasi
Ideasi, Curah Gagasan, & Pemotongan UI (UI Slicing)	Seri Google Gemini (Gemini 1.5 Pro, 2.0 Pro, 3.1 Pro).	Memiliki jendela konteks yang sangat masif dan kemampuan penalaran visual spasial yang superior. Ekstrem kreatif dalam

		menerjemahkan abstrak ke dalam struktur UI, animasi, dan tata letak modern yang kohesif.
Logika Kompleks, Arsitektur Backend, & Eksekusi	Seri Anthropic Claude (Claude 3.5 Sonnet, Claude 4.5 Opus).	Diakui sebagai "standar emas" untuk arsitektur pengkodean. Sangat deterministik dalam mengikuti struktur skema basis data, mendefinisikan API yang ketat, dan memelihara konsistensi di ribuan baris kode tanpa mengalami deviasi logika.
Pengujian, Pemecahan Bug Lingkungan Terisolasi, & Terminal	Model hemat biaya tinggi seperti DeepSeek atau seri Qwen .	Digunakan untuk tugas rutin yang berulang atau pemecahan log server. Menghemat konsumsi token komputasi yang mahal dari model Tier-1 tanpa mengorbankan akurasi operasional latar belakang.

Arsitektur Tumpukan Teknologi yang Dioptimalkan untuk AI (AI-Optimized Tech Stack)

Pemilihan kerangka kerja perangkat lunak (framework) tidak lagi didasarkan semata-mata pada preferensi pengembang, melainkan sangat dipengaruhi oleh jejak kehadiran data pelatihan model AI. Pengembang secara sengaja memilih teknologi yang "ramah AI" (AI-friendly)—artinya, teknologi ini memiliki jumlah dokumentasi, repositori GitHub sumber terbuka, dan data pelatihan yang sangat besar, sangat terstruktur, dan dapat diprediksi sehingga LLM memahaminya secara mendalam.¹

- **Arsitektur Antarmuka Pengguna (Frontend):** Kerangka kerja Next.js yang disandingkan dengan ekosistem React adalah pilihan dominan yang tak terbantahkan. Alasan utamanya adalah bahwa LLM dapat menghasilkan komponen fungsional dan perutean berbasis folder dalam Next.js dengan tingkat kepastian deterministik yang hampir absolut.
- **Paradigma Penataan Gaya (Styling):** Penggunaan Tailwind CSS diamankan secara

luas. Karena Tailwind bergantung pada kelas utilitas (utility classes) yang tertanam langsung di dalam elemen HTML atau JSX, model AI tidak diharuskan untuk memetakan aturan penataan gaya di seluruh file CSS eksternal yang terpisah. Hal ini secara drastis mengurangi komplikasi kognitif pada AI, meminimalkan kesalahan rendering, dan mencegah halusinasi penataan gaya. Selanjutnya, pustaka komponen UI pragmatis seperti Shadcn/ui diimplementasikan untuk menjamin estetika tingkat korporat dengan rekayasa instruksi (prompt engineering) yang sangat minim. Beberapa alur kerja juga menggunakan utilitas seperti Twix CN untuk injeksi tema.

- **Manajemen Basis Data & Relasi:** Kombinasi Drizzle ORM (Object-Relational Mapping) dengan basis data PostgreSQL (atau SQLite untuk versi ringkas) merupakan protokol standar industri baru. LLM terkadang kesulitan mengeksekusi sintaksis SQL mentah yang rumit dan rentan membuat kesalahan tipe data. Drizzle ORM menyelesaikan masalah ini dengan mendefinisikan skema dan kueri langsung dalam sistem TypeScript yang kuat dan ketat (type-safe). Lebih lanjut, hal ini memungkinkan AI mengelola migrasi basis data murni melalui modifikasi kode fungsional.
- **Infrastruktur Otentikasi:** Implementasi manual dari protokol otentikasi oleh agen AI sering kali memperkenalkan celah kerentanan keamanan yang parah dan logika sesi yang berliku-liku. Oleh karena itu, perpustakaan Better Auth direkomendasikan secara eksplisit dan universal. Better Auth menyediakan solusi prapaket yang komprehensif, aman, dan sangat kompatibel dengan arsitektur berbasis token yang dirancang AI.
- **Arsitektur Repositori:** Struktur Monorepo (sering diorkestrasi menggunakan Turbo Repo dan PNPM) adalah standar struktural terbaik. Konstruksi Monorepo memungkinkan agen AI untuk memuat konteks lingkungan frontend dan backend secara simultan dalam satu jendela analitis tunggal. Hal ini memastikan bahwa setiap perubahan kecil pada API secara otomatis tercermin dalam parameter pemanggilan di sisi klien.

Fase I: Arsitektur Dokumen Persyaratan Produk (PRD)

Dalam siklus pengembangan konvensional, *Project Requirements Document* (PRD) adalah artefak bisnis tingkat tinggi yang digunakan manajer proyek untuk berkomunikasi dengan pemangku kepentingan. Namun, dalam alur kerja yang sentris pada AI, fungsi PRD berevolusi menjadi jangkar pengekangan algoritmik. Ini adalah parameter konseptual pertama yang mencegah model bahasa dari deviasi ruang lingkup yang berbahaya. Jika PRD tidak jelas, semua turunan kode akan cacat secara logis.

Alur Kerja Formulasi PRD

Sintesis PRD selalu dimulai di luar IDE pengkodean utama untuk menjaga jendela konteks (context window) IDE tetap bersih dari kebisingan percakapan curah gagasan. Pengembang memanfaatkan asisten obrolan (chatbot) dengan kemampuan penalaran tinggi—seperti Gemini Pro dalam "mode berpikir" (thinking mode), ChatGPT, DeepSeek, atau alat analitis spesifik seperti ngodingpakai.com dan Kimi K2—untuk membentuk ide awal.

Untuk memastikan model bahasa memiliki referensi dunia nyata yang substansial, pengembang

modern menerapkan taktik ekstraksi. Mereka menggunakan alat perayap (web-scraping) dan penggali data seperti Firecrawl atau Deep Wiki untuk mencerna halaman landas produk yang sudah ada, repositori pesaing (pesaing seperti roadmap.sh), atau dokumentasi sumber terbuka. Data berstruktur longgar ini dimasukkan ke dalam LLM bersamaan dengan dorongan spesifik yang menuntut keluaran terstruktur tingkat militer.

Instruksi kritis pada tahap ini sangat eksplisit: **"Tulis PRD terlebih dahulu; jangan mulai menulis kode aplikasi apa pun"**. Dokumen PRD yang dihasilkan tidak langsung diterima. Operator manusia harus melakukan tinjauan teknis, menyesuaikan target pengguna, mengubah pedoman estetika (misalnya, menuntut antarmuka desain linier yang apik, "Linear-style", dengan penerapan gradien halus), dan memastikan definisi ruang lingkup *Minimum Viable Product* (MVP) sangat tajam. Setelah disempurnakan, keluaran ini disimpan secara permanen sebagai file bernama prd.md atau summary.md di direktori akar ruang kerja (workspace) agen.

Rancangan Cetak Biru Templat PRD (Dioptimalkan untuk AI)

Untuk meniru dan memanfaatkan alur kerja yang dijelaskan dalam penelitian ini, pengguna harus memasukkan instruksi ke AI untuk menghasilkan PRD dengan batasan struktural yang ketat. Berikut adalah formulasi arsitektur matriks untuk PRD yang dioptimalkan untuk injeksi konteks AI.

Bagian PRD	Fungsi dan Implikasi Psikologi Pemodelan LLM	Elemen Data Kritis yang Harus Didefinisikan
1. Ringkasan Eksekutif & Definisi Masalah	Membangun jangkar semantik awal. Mencegah algoritma halusinasi menciptakan fitur-fitur kompleks yang menyimpang dan tidak berkontribusi pada solusi masalah inti.	- Poin rasa sakit (pain point) pengguna. - Solusi fungsional inti yang diusulkan. - Demografi pengguna akhir secara spesifik.
2. Sistem Desain, Tema, & Estetika Visual	Mendikte batasan sintaksis antarmuka. Mencegah AI mencampuradukkan filosofi desain UI secara acak (misalnya, membuat panel gaya material-design di samping tata letak	- Palet warna dengan kode HEX yang ketat. - Tipografi utama (Spesifikasi Google Fonts). - Referensi filosofi visual (mis. "dark mode default", "efek

	neo-brutalism).	glassmorphism", "bento-grid layouts").
3. Cakupan Ruang Lingkup MVP (Minimum Viable Product)	Menegakkan batas eksekusi komputasi. Menginstruksikan kepada AI mengenai fitur apa saja yang tidak boleh dibangun saat ini, guna memusatkan sumber daya token.	- Daftar periksa fitur inti yang tidak dapat dinegosiasikan. - Daftar pengecualian absolut (Peta Jalan Masa Depan).
4. Aliran Aplikasi (User Flow) & Perjalanan Utama	Menyediakan landasan logika topologis untuk tata letak perutean (routing). Memungkinkan AI untuk memprediksi transisi status antara layar sebelum layar tersebut dikodekan.	- Arsitektur pendaftaran (onboarding). - Putaran aksi inti (misalnya, "Pengguna masuk otentikasi -> melihat dasbor -> mengelola inventaris").
5. Pedoman Tumpukan Teknologi Awal (Pre-Stack Spec)	Meskipun sebagian besar akan ditangani oleh TSD, spesifikasi tumpukan tingkat tinggi mencegah ideasi yang bertentangan di agen lain.	- Pilihan kerangka kerja web dan bahasa pemrograman (Next.js, TypeScript).

Fase II: Spesifikasi Fungsional Perangkat Lunak (FSD)

Sementara PRD mendefinisikan *apa* entitas aplikasi itu, *Functional Specification Document* (FSD) menentukan secara tepat *bagaimana* aplikasi tersebut akan beroperasi, baik secara logis, mekanis, maupun presentasi visual. Berbeda dengan metodologi warisan yang usang, FSD dalam alur kerja spesifikasi-AI jarang merupakan dokumen statis yang ditulis secara independen oleh manusia. FSD secara dinamis disintesis oleh alat komputasi di dalam IDE, menggunakan entitas prd.md yang telah divalidasi sebagai materi benih masukannya (input seed).

Mekanisme Eksekusi "Mode Perencanaan" (Plan Mode)

Untuk memulai sintesis FSD, pengembang manusia mengaktifkan agen IDE (seperti Anti Gravity) dan melampirkan file prd.md yang disetujui sebelumnya secara langsung ke jendela antarmuka instruksi. Menggunakan perintah eksplisit atau pintasan papan tik tingkat lingkungan (seperti Shift + Tab), pengembang memicu "Mode Perencanaan" atau menuntut "Rencana Implementasi" (Implementation Plan) dari model AI (dengan Claude Opus sangat disarankan untuk fase rasionalisasi struktur ini).

Agen kecerdasan buatan akan mengurai abstrak bisnis menjadi serangkaian langkah fungsional, komponen logika, dan jalur aliran data yang konkret. Di sinilah gaya arsitektur yang canggih sangat penting: alur kerja berkualitas tinggi secara spesifik mewajibkan AI untuk mengintegrasikan bahasa diagram sintaksis, khususnya *Mermaid Diagrams*, langsung ke dalam struktur penurunan (markdown) FSD.

Dengan memaksa model bahasa untuk menghasilkan diagram urutan UML (Unified Modeling Language) visual dan diagram alur grafik topologi, pengembang manusia dapat secara visual memverifikasi pemahaman topologi mendasar AI mengenai manajemen status (state management) dan arah perpindahan data, jauh sebelum siklus pemrosesan komputasi dihabiskan untuk merender kode nyata.

Setelah Rencana Implementasi (FSD) dipublikasikan oleh agen, pengembang memulai peninjauan kolaboratif menggunakan fitur "anotasi sebaris" (inline commenting). Jika AI berencana untuk membangun manajemen status global yang sangat kompleks (misalnya menggunakan Redux) untuk fitur antarmuka yang sangat mendasar, operator akan menyuntikkan instruksi umpan balik pada baris tertentu dari rencana tersebut, memerintahkan AI untuk mengecilkan solusi teknisnya, misalnya dengan beralih menggunakan *React Context API* atau *TanStack Query* secara lokal. Proses anotasi Rencana Implementasi ini pada dasarnya adalah kompilasi FSD yang telah diuji oleh batas.

Rancangan Cetak Biru Templat FSD (Dioptimalkan untuk AI)

Untuk mengatur alur kerja pengkodean arsitektural yang kohesif, instruksi (prompt) manusia kepada asisten perencanaan harus secara eksplisit mewajibkan keluaran dokumen FSD yang sesuai dengan kerangka kerja di bawah ini.

Bagian FSD	Fungsi dan Implikasi Psikologi Pemodelan LLM	Elemen Data Kritis yang Harus Didefinisikan
1. Gambaran Umum Arsitektur & Hierarki Pohon Komponen	Menginstruksikan struktur navigasi mental AI. Memastikan model tidak membangun aplikasi monolitik raksasa, melainkan	- Pemetaan rinci struktur direktori (/app, /components, /lib). - Pemisahan masalah yang

	pohon komponen modular.	eksplisit (Pemisahan antara layanan data vs. rute interaksi antarmuka).
2. Cetak Biru Anatomi Komponen Antarmuka Pengguna	Menentukan secara granular variabel dan elemen dari setiap interaksi layar. Menjadi prasyarat mutlak sebelum eksekusi pemotongan UI otomatis ("UI Slicing").	- Analisis tata letak layar demi layar (screen-by-screen breakdown). - Penetapan spesifik pustaka komponen (mis., Tabel dari Shadcn, transisi dari Framer Motion).
3. Alur Data Topologi & Manajemen Status (State Management)	Mencegah kesalahan logika pengikatan data (race conditions) dan ketidakkonsistenan status dengan membatasi di mana dan bagaimana status aplikasi dimutasikan.	- Integrasi diagram alur berbasis Mermaid.js & Diagram Urutan UML yang merinci transisi dari lapisan presentasi ke logika inti.
4. Matriks Rencana Implementasi Bertahap (Task Execution List)	Alat komputasi untuk membongkar tugas besar. Penting untuk mengakomodasi model dengan jendela batas token kecil. Menjaga AI agar tidak kehilangan fokus historis.	- Langkah 1: Inisialisasi struktur kerangka kerja. - Langkah 2: Pengembangan cangkang luar UI & navigasi. - Langkah n: Pemetaan tiruan data (dummy data mapping).

Fase III: Dokumen Spesifikasi Teknis (TSD) dan Kontrak Server

Technical Specification Document (TSD) mewakili konfigurasi arsitektur paling tidak fleksibel, sangat deterministik, dan bebas kompromi di seluruh siklus pengembangan kecerdasan buatan. Apabila FSD mendikte pengalaman operasional dan logika visual untuk sisi klien (frontend), maka TSD memegang kekuasaan penuh atas integrasi tulang punggung (backend),

topologi mekanika basis data, dan orkestrasi panggilan antarmuka pemrograman aplikasi (API).

Penerapan Logika "Kontrak API" dan Penyelarasan Skema

Secara inheren, model bahasa besar merupakan struktur matematis terisolasi yang tidak memiliki memori spasial jangka panjang, serta tidak memiliki kesadaran otomatis mengenai status sistem dari program yang dibangunnya di area lain. Jika, misalnya, agen AI yang bertugas membuat antarmuka mengirimkan instruksi muatan JSON, agen yang menyusun kode basis data tidak dapat menerima instruksi tersebut jika tidak terdapat landasan referensial bersama. Absennya "Kontrak API" bersama ini berpotensi menggagalkan komunikasi inter-sistemik dan membuat aplikasi runtuh seketika.

Oleh karena itu, metodologi TSD mutakhir mendikte pengembang manusia untuk mengarahkan model penalaran tingkat tinggi (Claude Opus 4.5 direkomendasikan secara khusus di sini) guna membentuk file statis terpisah: `api-reference.md`, `backend-plan.md`, atau *API Contract*. File ini merupakan cetak biru terperinci dari relasi data. Di sinilah Drizzle ORM sangat penting; pengembang akan memaksa AI untuk mensintesis seluruh tabel data, relasi tabel bersarang (nested relationships), kunci primer (primary keys), serta jenis deklarasi variabelnya, semuanya dalam bahasa TypeScript standar yang dimengerti Drizzle.

Hal yang lebih kompleks seperti logika kontrol akses berbasis hak istimewa pengguna didefinisikan di sini. TSD harus menjabarkan dengan tepat titik persimpangan otorisasi API mana yang terekspos ke ruang publik, titik akhir apa yang hanya merespons token sesi aman yang valid, dan bagaimana hak asuh diferensial beroperasi, seperti pemisahan entitas batas penagihan logis antara tipe pengguna "Dasar" (Free) dengan entitas "Premium" (Pro). Keberadaan file spesifik ini membentuk batas deterministik dan menjamin bahwa agen yang menghasilkan kode backend tidak berhalusinasi menghasilkan baris atribut relasi SQL yang cacat.

Rancangan Cetak Biru Templat TSD (Dioptimalkan untuk AI)

Eksekusi tulang punggung komputasi harus tanpa cela. Ketika merumuskan instruksi untuk sintesis TSD, pengembang harus mengharuskan kecerdasan buatan memasukkan blok logika struktural matriks berikut ini.

Bagian TSD	Fungsi dan Implikasi Psikologi Pemodelan LLM	Elemen Data Kritis yang Harus Didefinisikan
1. Matriks Tumpukan Teknologi Backend yang Kaku	Deklarasi otoritatif lingkungan. Apabila disinergikan dengan server Model Context Protocol (MCP) seperti Context7, ini	- Pemilihan sistem <i>Runtime</i> dan kerangka kerja server (Node.js, Express.js, Hono). - Pewajiban eksekusi

	memaksa AI menggunakan versi pustaka spesifik, menghilangkan ancaman penggunaan kode API usang.	TypeScript untuk menjaga kekakuan skema. - Resolusi arsitektur integrasi otentikasi (Better Auth).
2. Rekayasa Topologi Skema Basis Data (Database Schema)	Satu-satunya sumber kebenaran (Source of Truth) matematis untuk operasi persistensi data.	- Deklarasi entitas arsitektur model menggunakan sintaks Drizzle ORM. - Pemetaan eksplisit konfigurasi Relasi (<i>One-to-Many</i> , <i>Many-to-Many</i>). - Implementasi aturan validasi konseptual untuk mencegah injeksi data kotor.
3. Kontrak Negosiasi Titik Akhir API (The API Contract)	Mengikat modul sisi klien (Frontend) dengan server. Mengunci ekspektasi input-output algoritme AI.	- Penamaan URL rute dan hierarki struktur titik akhir (misal, <code>/api/v1/orders</code>). - Ekspektasi Metode Protokol Transfer (GET, POST, PATCH, DELETE). - Model spesifik muatan permintaan dan parameter tanggapan struktural JSON.
4. Logistik Perutean Webhook dan Transaksi Eksternal	Krusial saat mengatur panggilan interaktif dengan entitas komputasi pihak ketiga (misalnya, penyelesaian gerbang pembayaran komersial seperti Midtrans). Membangun topologi	- Mekanika resolusi persetujuan untuk pengamanan rute Webhook berbasis <i>Signature Verification</i> . - Penanganan kegagalan otomatis untuk batas laju API eksternal.

	keamanan siber.	- Parametrik konfigurasi terowongan proksi (proxy tunneling) menggunakan layanan ngrok guna pengujian respons lingkungan lokal yang aman.
--	-----------------	---

Metodologi Eksekusi Praktis dan Rekayasa Pengujian Otomatis

Dengan dokumen arsitektural—PRD, FSD, dan TSD—terverifikasi, dikompilasi ke dalam format *Markdown* (.md), dan disematkan langsung dalam struktur akar direktori pengembangan lokal, insinyur dapat melepaskan eksekusi penulisan kode aktif. Gaya pengembangan tingkat lanjut ini secara inheren menolak pembuatan blok sistem ganda secara simultan. Gaya tersebut memaksakan strategi pemrosesan linier bertahap (phased development), yang dioptimalkan dalam pendekatan spesifik "Pengutamaan Antarmuka Pengguna" (Frontend-First Strategy).

Gaya Implementasi Berfokus Antarmuka dan Sintesis Pola (Frontend-First)

Aturan operasional pertama dalam mengarahkan agen kecerdasan buatan berfokus sepenuhnya pada arsitektur antarmuka sebelum mengalokasikan siklus pertimbangan apa pun pada abstraksi basis data atau pemetaan rute API backend. Mendorong kompleksitas sinkronisasi lapisan komputasi basis data bersamaan dengan desain responsif ke dalam satu permintaan jendela interaktif tunggal adalah titik jenuh operasional yang secara universal menginduksi halusinasi tingkat lanjut dan ambruknya kognisi pada agen bahasa sistem dasar.

Pengembang perangkat lunak menuntut agen IDE (seperti Claude 3.5 Sonnet dalam perangkat lunak Anti Gravity) untuk memotong bagian UI (UI Slicing) secara terisolasi dengan mengacu pada panduan visual FSD. Taktik manipulasi konteks yang sangat penting pada tahap ini adalah instruksi absolut untuk menerapkan "Data Dummy" di seluruh halaman. Logika ini memungkinkan agen berfokus murni pada resolusi rendering *Next.js*, merangkai variabel komponen penataan *Tailwind*, serta merender tata letak kompleks berbasis *Bento grid* atau animasi *Framer Motion*. Keberhasilan visual ini memungkinkan para insinyur tingkat atas menjalankan analisis jaminan kualitas antarmuka secara waktu nyata (real-time feedback loop), menghindari masa tunggu panjang yang diperlukan untuk interkoneksi integrasi basis data.

Koreksi antarmuka dilakukan secara iteratif menggunakan rekayasa petunjuk diskret lokal. Berbeda dengan pendekatan manual mengedit ratusan baris file CSS atau parameter dalam *divs HTML*, insinyur hanya mengeluarkan instruksi bahasa alami yang ringkas—contoh: "ubah komponen navigasi samping (sidebar) menjadi spesifikasi ringkas, pertahankan status

transparan aslinya," atau "implementasikan komponen desain animasi grid modern pada parameter bagian pahlawan (hero section)".

Konvergensi Integrasi Basis Data dan Mekanika Penelusuran Serangga Otonom (Autonomous Debugging)

Ketika lapisan antarmuka pengguna telah mencapai keseimbangan presisi dan parameter visibilitas telah disetujui, siklus beralih ke aktivasi rekayasa backend. Insinyur sistem lalu merujuk agen kembali kepada "Kontrak API" spesifik di dalam TSD. Agen sekarang mengubah blok referensi data tiruan aslinya menjadi fungsi injeksi kerangka kerja asinkronus aktif, sering kali diorkestrasi melalui model sinkronisasi *TanStack Query* yang menjembatani klien dengan logika rute API internal. Dalam ekosistem yang terisolasi, arsitektur skema basis data Drizzle menghasilkan skrip inisiasi yang menyemai baris data acak yang dapat dimanipulasi dengan cepat (`npm run db:seed`), membuat sistem siap operasional dalam hitungan jam, bukan minggu.

Namun, di tengah interkoneksi antara kerangka verifikasi klien, fungsi penguraian otentikasi berbasis token dari *Better Auth*, dan penyetelan muatan basis data PostgreSQL, insiden fatal logika tidak dapat dihindari. Di sinilah metodologi analitik dan resolusi kesalahan otonom log interaktif beroperasi penuh.

Pengembang tidak membuang waktu menafsirkan ratusan garis rantai penelusuran balik memori (stack traces log). Mereka menyalin seluruh blok jejak eksekusi yang menghasilkan kegagalan *Terminal Output*, atau menyoroti secara semantik komponen skrip yang rentan, lalu mengembalikannya langsung ke antarmuka komunikasi penyelesaian agen utama IDE. Agen, karena memiliki visibilitas sempurna atas sumber kebenaran direktori aslinya, serta akses komputasi tingkat server dokumentasi pembaruan parameter MCP terbaru, secara intuitif menelusuri perbedaan struktur dalam konfigurasi jembatannya, menyusun skenario kesalahan, lalu mempublikasikan ulang arsitektur solusi sistem kode dengan memintasi operator manusia sepenuhnya. Pendekatan perputaran otomatis "Self-Healing Debugging Loop" (Log-Based Debugging) ini adalah inovasi paling kuat dari metodologi siklus ini.

Inovasi Arsitektur Spesifikasi Produk (PSD) dan Verifikasi Kualitas Terdistribusi

Fase terminal transisi dari produksi ke verifikasi aplikasi meninggalkan teknik peninjauan manusia manual demi adopsi agen simulasi inspeksi kualitas kecerdasan terdistribusi, menggunakan model pengujian lingkungan berorientasi peramban interaktif terdepan seperti TestSprite (yang sangat bergantung pada server pengujian integrasi berbasis integrasi MCP).

Untuk melengkapi instrumen evaluasi independen tersebut, direktur teknis manusia memfungsikan agen konstruktor primer sistem mereka dengan tugas turunan analitik akhir: Membuat instrumen Spesifikasi Produk Definitif yang dikenal luas sebagai *Product Specification Document* (PSD). Jika dokumen tahap PRD memegang kendali teoritis terhadap fitur perangkat lunak masa depan, PSD berdiri secara retrospektif mengodifikasi peta lengkap

infrastruktur mekanik akhir dari perutean yang divalidasi dan dibangun secara operasional. Hal ini secara struktural bertindak sebagai pelacak koordinat ruang spasial fungsional.

Data parameter arsitektural PSD lalu digunakan sebagai referensi pengarah simulasi di dalam otak algoritme komputasi platform eksternal. Secara otonom memanipulasi utilitas perangkat lunak nir-antarmuka (headless browser instance) dari platform Chrome, agen inspeksi kualitas menjalankan protokol masuk pendaftaran tiruan (dummy account registration), mengklik dan mentransisikan vektor kursor komponen tombol UI grafis visual, serta mengartikulasikan skenario perilaku inti ekstensif dari siklus jalur perjalanan yang dijanjikan pada dokumen PRD. Apabila salah satu variabel metrik mengembalikan penolakan tanggapan muatan kegagalan—misalnya "Kode Pengujian Kasus Metrik TC024 merekam kehabisan waktu waktu server kesalahan 500"—entitas uji ini menurunkan hasil laporan pelacakan. Laporan struktural diintegrasikan ulang oleh insinyur kembali sebagai instruksi paksaan di IDE eksekusi basis pengkodean utama, mengamankan simpul pemulihan arsitektur melingkar sempurna untuk perbaikan tanpa sentuhan komputasi manusia.

Infrastruktur sistem yang disetujui akhirnya disebarkan ke instrumen eksekusi arsitektur terdistribusi *Cloud-Native*, memastikan pemrosesan klien sisi repositori terintegrasi dengan alat penerapan layanan mikro langsung perantara sistem *Vercel* atau *Cloudflare*, sementara topologi simpul lapisan data Docker dan PostgreSQL dikandung secara virtual melalui konfigurasi server pribadi (VPS) melalui sistem pengaturan infrastruktur *Coolify*.

Kesimpulan

Siklus arsitektural generasi peranti lunak berbasis kecerdasan mesin menghadirkan perubahan radikal dari struktur teknis tradisional manual kepada spesialisasi kontrol manajemen parameter. Tingkat kesuksesan operasional dari komputasi asisten IDE bergantung sepenuhnya pada kesempurnaan struktural dokumen orientasi spesifikasi dan penguatan batas algoritmik dari pengembang awal komputasi.

Dengan mengangkat *Project Requirements Document* (PRD) dari fungsi komunikasi deskriptif bisnis menjadi penahan logis, memajukan metodologi desain operasional *Functional Specification Document* (FSD) kepada integrasi parameter topologi, dan menyusun spesifikasi ketat *Technical Specification Document* (TSD) untuk membentuk keselarasan pertukaran antarmuka sisi server bebas kontradiksi, pengembang teknis memformulasi struktur di mana model bahasa canggih berjalan otonom dan aman dari ancaman defleksi intelektual palsu, atau masalah halusinasi logis.

Injeksi metode integrasi format berkas turunan tipe *Markdown* terenkripsi dalam cakupan area ruang sistem asisten pengkodean agen utama, serta penerapan parameter model komputasi protokol konteks perantara *Retrieval-Augmented Generation* dan adopsi penyaringan strategis dari eksekusi pembangunan antarmuka awal bertahap terisolasi, merepresentasikan standar industri paling elit dari produksi perangkat lunak fungsional terkini. Implementasi cetak biru dokumentasi yang disajikan menjanjikan kemampuan skalabilitas penuh kepada integrator

teknologi masa depan.

Karya yang dikutip

1. Workflow AI coding TERBAIK! Udah bantu gua bikin puluhan aplikasi yang menghasilkan CUAN \$\$\$, diakses April 9, 2026,
<https://www.youtube.com/watch?v=yoVo-iOCO4Q>